Name: Joe Merrill
Date: 3 FEB 2026
Week: 4 - Create a Design Model for a Small Bed & Breakfast Reservation System

## 1) Rubric Criteria: Create software system architecture diagram 10%

## Your Response:

```
SMALL B&B RESERVATION SYSTEM - SOFTWARE ARCHITECTURE DIAGRAM
============================================================

3-Tier Layered Architecture

┌──────────────────────────────────────────────────────────────────┐
│                    SMALL B&B RESERVATION SYSTEM                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                   PRESENTATION LAYER                     │   │
│  │                       (User Interface)                   │   │
│  ├──────────────────────────────────────────────────────────┤   │
│  │  ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐ │   │
│  │  │   Guest      │ │    Room      │ │    Reservation   │ │   │
│  │  │ Management   │ │ Management   │ │    Management    │ │   │
│  │  │   Screen     │ │   Screen     │ │      Screen      │ │   │
│  │  └──────────────┘ └──────────────┘ └──────────────────┘ │   │
│  │  ┌──────────────┐ ┌──────────────┐                      │   │
│  │  │  Calendar    │ │   Payment    │                      │   │
│  │  │    View      │ │   Screen     │                      │   │
│  │  └──────────────┘ └──────────────┘                      │   │
│  └──────────────────────────────────────────────────────────┘   │
│                              │                                   │
│                              ▼                                   │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                  BUSINESS LOGIC LAYER                    │   │
│  │                   (Application Services)                 │   │
│  ├──────────────────────────────────────────────────────────┤   │
│  │  ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐ │   │
│  │  │   Guest      │ │    Room      │ │    Reservation   │ │   │
│  │  │  Service     │ │   Service    │ │     Service      │ │   │
│  │  └──────────────┘ └──────────────┘ └──────────────────┘ │   │
│  │  ┌──────────────┐ ┌──────────────┐                      │   │
│  │  │  Calendar    │ │   Payment    │                      │   │
│  │  │  Service     │ │   Service    │                      │   │
│  │  └──────────────┘ └──────────────┘                      │   │
│  └──────────────────────────────────────────────────────────┘   │
│                              │                                   │
│                              ▼                                   │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                   DATA ACCESS LAYER                      │   │
│  │                     (Persistence)                        │   │
│  ├──────────────────────────────────────────────────────────┤   │
│  │  ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐ │   │
│  │  │   Guest      │ │    Room      │ │    Reservation   │ │   │
│  │  │ Repository   │ │ Repository   │ │    Repository    │ │   │
│  │  └──────────────┘ └──────────────┘ └──────────────────┘ │   │
│  │  ┌──────────────────────────────────────────────────────┐   │
│  │  │              Payment Repository                      │   │
│  │  └──────────────────────────────────────────────────────┘   │
│  └──────────────────────────────────────────────────────────┘   │
│                              │                                   │
│                              ▼                                   │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                       DATABASE                           │   │
│  │                  (Local SQLite/File)                     │   │
│  │  ┌──────────────┐ ┌──────────────┐ ┌──────────┐ ┌──────────┐ │   │
│  │  │   Guests     │ │    Rooms     │ │Reserva-  │ │Payments  │ │   │
│  │  │   Table      │ │    Table     │ │ tions    │ │  Table   │ │   │
│  │  └──────────────┘ └──────────────┘ └──────────┘ └──────────┘ │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘


ARCHITECTURE COMPONENTS SUMMARY
===============================

Layer               | Purpose                        | Components
--------------------|--------------------------------|----------------------------------
Presentation        | User interface for John/Jane   | Guest, Room, Reservation,
                    |                                | Calendar, Payment screens
--------------------|--------------------------------|----------------------------------
Business Logic      | Application rules & validation | Services handling availability
                    |                                | checks, payment guarantees,
                    |                                | reservation drops
--------------------|--------------------------------|----------------------------------
Data Access         | Database operations            | Repositories for CRUD operations
--------------------|--------------------------------|----------------------------------
Database            | Persistent storage             | Local database (SQLite or similar)
```

**2) Rubric Criteria: Explain approach, steps, and rationale of the software architecture diagram 20%**
**Your Response:**

The design approach followed a requirements driven methodology, analyzing the business needs of John and Jane's B&B to select an architecture that matches the system's scale, complexity, and usage patterns.

Steps Taken
1. Analyze System Requirements
-Small scale, single location business
- 2 users (John and Jane) - no guest self-service
-Core functions: manage guests, rooms, reservations, and payments
-Need for persistent data storage
-No requirement for web access or distributed computing

2. Evaluate Architecture Pattern
-3 tier layered pattern
-Positives: Separation of concerns, maintainable
-Negatives: Slightly more complex

3. Define Layers
-Mapped each requirement to the appropriate layer
-Ensured single responsibility principle on each layer

Step 4: Identify Components Within Each Layer
-Derived components directly from the analysis class diagram (Guest, Room, Reservation, Payment, Calendar)

## 3) Rubric Criteria: Create detailed UML class diagram 15%

## Your Response:

```
SMALL B&B RESERVATION SYSTEM - DETAILED UML CLASS DIAGRAM
=========================================================

┌──────────────────────────┐          ┌──────────────────────────┐
│          Guest           │          │         Calendar         │
├──────────────────────────┤          ├──────────────────────────┤
│ - guestId: int           │          │ - calendarId: int        │
│ - firstName: String      │          │ - year: int              │
│ - lastName: String       │          │ - month: int             │
│ - address: String        │          ├──────────────────────────┤
│ - city: String           │          │ + getAvailableRooms(start:  │
│ - state: String          │          │     Date, end: Date): Room[] │
│ - zipCode: String        │          │ + checkAvailability(roomId: │
│ - phoneNumber: String    │          │     int, start: Date,    │
│ - creditCardNumber: String │        │     end: Date): boolean  │
│ - creditCardExpiry: Date │          │ + blockDates(roomId: int,   │
├──────────────────────────┤          │     start: Date,         │
│ + createGuest(): void    │          │     end: Date): void     │
│ + updateGuest(): void    │          │ + releaseDates(roomId: int, │
│ + deleteGuest(): void    │          │     start: Date,         │
│ + getGuestById(id: int):  │         │     end: Date): void     │
│     Guest                │          └──────────────────────────┘
│ + searchGuest(name: String):  │                    │
│     Guest[]              │                    │ 1
│ + validateCreditCard():  │                    │
│     boolean              │                    │
└──────────────────────────┘                    │
            │                                   │
            │ 1                                 │
            │                                   │
            │ makes                             │ shows
            │                                   │ availability
            │ *                                 │
            ▼                                   │
┌──────────────────────────┐        1       *  │
│        Reservation       │◇──────────────────────────────────┐
├──────────────────────────┤                   │               │
│ - reservationId: int     │                   │               │
│ - guestId: int           │                   │               │
│ - roomId: int            │                   ▼               │
│ - startDate: Date        │          ┌──────────────────────────┐
│ - endDate: Date          │          │          Room            │
│ - totalPrice: decimal    │          ├──────────────────────────┤
│ - guaranteeDeadline: Date │         │ - roomId: int            │
│ - isGuaranteed: boolean  │          │ - roomNumber: String     │
│ - status: ReservationStatus │       │ - roomType: String       │
│ - createdDate: DateTime  │          │ - pricePerNight: decimal │
├──────────────────────────┤          │ - description: String    │
│ + createReservation(): void │       │ - isActive: boolean      │
│ + cancelReservation(): void │       ├──────────────────────────┤
│ + updateReservation(): void │       │ + getRoomById(id: int): Room │
│ + getReservationById(id:  │         │ + getAllRooms(): Room[]  │
│     int): Reservation    │          │ + updateRoom(): void     │
│ + checkGuaranteeStatus():  │        │ + calculatePrice(nights:  │
│     boolean              │          │     int): decimal        │
│ + dropExpiredReservations():  │     └──────────────────────────┘
│     void                 │                   ◇
│ + getReservationsByDate(  │                   │
│     date: Date):         │          ┌────────┴────────┐
│     Reservation[]        │          │                 │
│ + getReservationsByGuest(  │        │                 │
│     guestId: int):       │          │                 │
│     Reservation[]        │  ┌──────────┐  ┌──────────┐  ┌──────────┐
└──────────────────────────┘  │RoomWithBath│ │RoomWithView│ │NormalRoom│
            │                 ├──────────┤  ├──────────┤  ├──────────┤
            │                 │-hasTub:  │  │-viewType: │  │          │
            │                 │ boolean  │  │ String   │  │          │
            │ 1               │-hasShower: │ │-floor:   │  │          │
            │                 │ boolean  │  │ int      │  │          │
            │                 ├──────────┤  ├──────────┤  ├──────────┤
            │ has             │+getAmenity │ │+getView  │  │          │
            │                 │ List():  │  │ Descrip- │  │          │
            │ 0..1            │ String[] │  │ tion():  │  │          │
            ▼                 └──────────┘  │ String   │  └──────────┘
┌──────────────────────────┐                └──────────┘
│         Payment          │
├──────────────────────────┤
│ - paymentId: int         │          ┌──────────────────────────┐
│ - reservationId: int     │          │    <<enumeration>>       │
│ - amount: decimal        │          │    ReservationStatus     │
│ - paymentDate: DateTime  │          ├──────────────────────────┤
│ - paymentType: PaymentType │        │ PENDING                  │
│ - creditCardLast4: String │         │ GUARANTEED               │
│ - transactionId: String  │          │ CHECKED_IN               │
│ - isRefunded: boolean    │          │ COMPLETED                │
├──────────────────────────┤          │ CANCELLED                │
│ + processPayment(): boolean │       │ DROPPED                  │
│ + refundPayment(): boolean │        └──────────────────────────┘
│ + getPaymentById(id: int):  │
│     Payment              │          ┌──────────────────────────┐
│ + getPaymentsByReservation(  │      │    <<enumeration>>       │
│     resId: int): Payment[] │        │      PaymentType         │
│ + calculateGuaranteeAmount(  │      ├──────────────────────────┤
│     pricePerNight: decimal): │      │ GUARANTEE                │
│     decimal              │          │ FULL_PAYMENT             │
└──────────────────────────┘          │ REFUND                   │
                                      └──────────────────────────┘

RELATIONSHIPS SUMMARY
=====================

Relationship              | Type          | Multiplicity
--------------------------|---------------|--------------------
Guest --> Reservation     | Association   | 1 Guest : * Reservations
Reservation --> Room      | Association   | * Reservations : 1 Room
Reservation --> Payment   | Composition   | 1 Reservation : 0..1 Payment
Room --> Calendar         | Association   | * Rooms : 1 Calendar
RoomWithBath --> Room     | Inheritance   | extends
RoomWithView --> Room     | Inheritance   | extends
NormalRoom --> Room       | Inheritance   | extends


UML NOTATION KEY
================

Symbol    | Meaning
----------|--------------------------------
-         | Private attribute/method
+         | Public attribute/method
#         | Protected attribute/method
◇         | Inheritance (arrow points to parent)
◆         | Composition (Filled diamond)
◇         | Aggregation (hollow diamond)
-->       | Association (dependency)
*         | Many (zero or more)
1         | Exactly one
0..1      | Zero or one (optional)
```

**4) Rubric Criteria: Explain approach, steps, and rationale of the detailed class diagram model 25%**
**Your Response:**

The class diagram was developed by refining the analysis class diagram. Each class was expanded with specific attributes and operations needed to fulfill the use cases.

1. Identify Attributes from Requirements

| Requirement | Attributes |
|---|---|
| Enter customer's name, address, phone number | Guest: firstName, lastName, address, city, state, zipCode, phoneNumber |
| Credit card number | Guest: creditCardNumber, creditCardExpiry |
| Reservation dates | Reservation: startDate, endDate |
| Room price | Room: pricePerNight; Reservation: totalPrice |
| Guaranteed by 1 day's payment | Reservation: isGuaranteed, guaranteeDeadline |
| Room with private bath, lake view, normal | Room subclasses with specific attributes |
| Reservations will be dropped | Reservation: status (enum with DROPPED state) |

2. Assign Methods Based on Use Cases

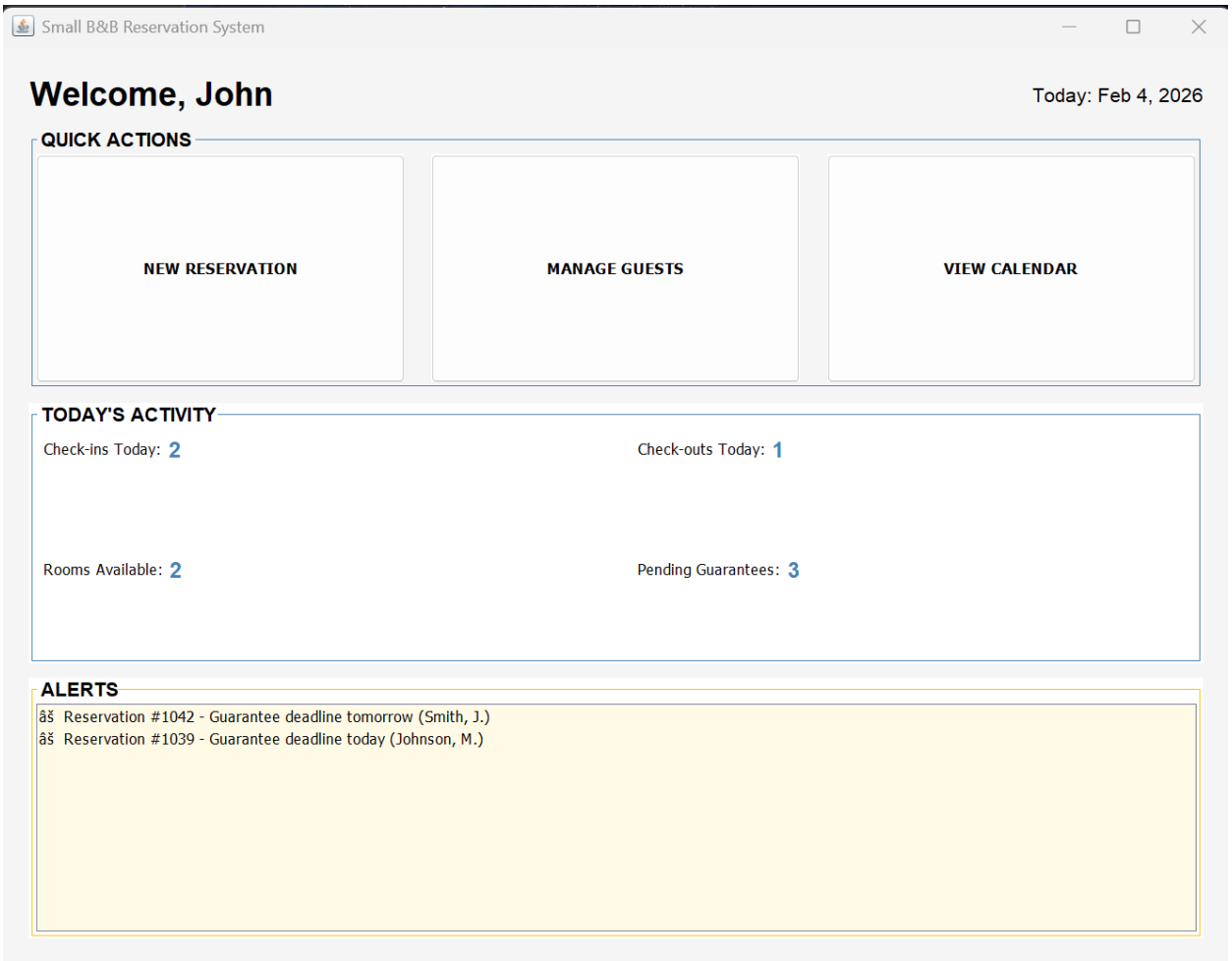
| Use Case | Methods Assigned |
|---|---|
| Maintain Guest Records | Guest: createGuest(), updateGuest(), deleteGuest(), getGuestById(), searchGuest() |
| Maintain Room Records | Room: getRoomById(), getAllRooms(), updateRoom(), calculatePrice() |
| Make Room Reservation | Reservation: createReservation(), updateReservation(); Calendar: getAvailableRooms(), checkAvailability(), blockDates() |
| Drop Room Reservation | Reservation: cancelReservation(), dropExpiredReservations(), checkGuaranteeStatus(); Calendar: releaseDates() |
| Manage Payment | Payment: processPayment(), refundPayment(), calculateGuaranteeAmount() |

3. Define Relationships and Multiplicities

| Relationship | Analysis | Design |
|---|---|---|
| Guest ↔ Reservation | A guest can have multiple reservations over time | 1 Guest : * Reservations |
| Reservation ↔ Room | Each reservation is for exactly one room | * Reservations : 1 Room |
| Reservation ↔ Payment | A guaranteed reservation has one payment; unguaranteed has none | 1 Reservation : 0..1 Payment |
| Room ↔ Calendar | All rooms share one calendar for availability | * Rooms : 1 Calendar |

4. Apply Inheritance for Room Types
From requirements: "one room with private bath... room with lake view... two normal rooms"
Room (Abstract Parent)
- LRoomWithBath (hasTub, hasShower, getAmenityList())
- LRoomWithView (viewType, floor, getViewDescription())
- LNormalRoom (no additional attributes)

5) Rubric Criteria: Create user interface mockup 10%
Your Response:



Small B&B Reservation System

< Back

## NEW RESERVATION - Step 1 of 3

Select Dates

Check-in Date: 01 / 01 / 2026

Check-out Date: 01 / 01 / 2026

Number of Nights: 3

FEBRUARY 2026

| Sun | Mon | Tue | Wed | Thu | Fri | Sat |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |

Cancel

Small B&B Reservation System

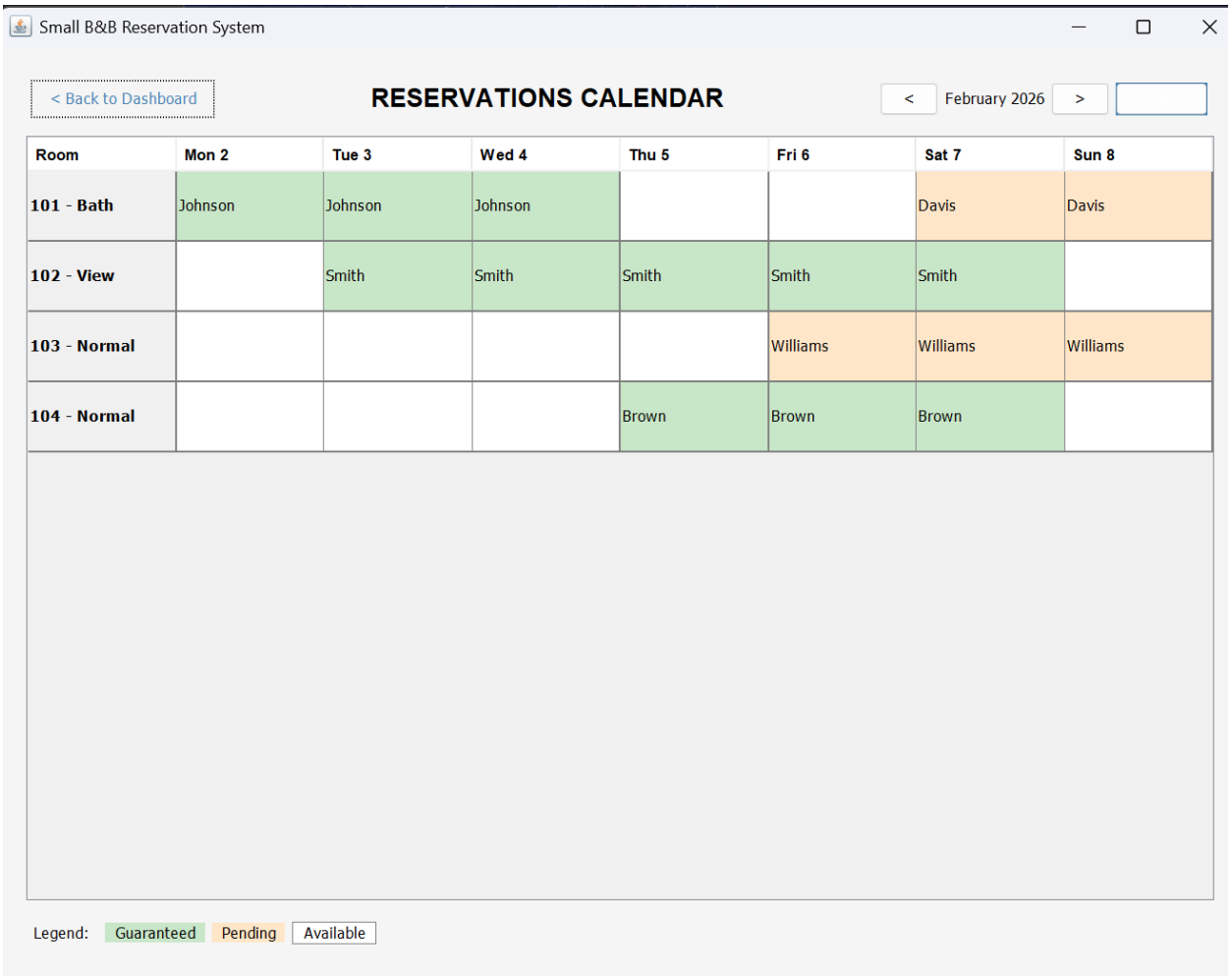
< Back

## MANAGE GUESTS

Search: Search

| Name | Phone | City | Reservations | Action |
|---|---|---|---|---|
| Smith, John | (617) 555-1234 | Boston, MA | 3 | Edit |
| Johnson, Mary | (212) 555-5678 | New York, NY | 1 | Edit |
| Williams, Bob | (415) 555-9012 | San Francisco, CA | 2 | Edit |
| Brown, Alice | (303) 555-3456 | Denver, CO | 1 | Edit |
| Davis, Carol | (617) 555-7890 | Boston, MA | 5 | Edit |

<< Prev Page 1 of 3 Next >>




**6) Rubric Criteria: Explain approach, steps, and rationale of the user interface mockup 10%**
**Your Response:**

The UI design followed a task centered design methodology It focuses on the primary users (John and Jane) and their main workflow: making reservations for guests who call by phone. The design prioritizes simplicity, efficiency, and error prevention.

Steps Taken
1. Identify User Tasks from Use Cases

| Use Case | UI Screens Required |
|---|---|
| Make Room Reservation | Date Selection → Room Selection → Guest/Payment → Confirmation |
| Maintain Guest Records | Guest Management Screen |
| Maintain Room Records | Calendar View (shows room status) |
| Drop Room Reservation | Dashboard Alerts + Calendar View |
| Manage Payment | Integrated into Reservation Flow |

2. Design the Navigation Flow

```
Dashboard (Home)
  |--New Reservation (Wizard: 3 steps)
    |---Step 1: Select Dates
    |---Step 2: Select Room
    |---Step 3: Guest Info & Payment
    |----Confirmation
   |--Manage Guests (CRUD operations)
   |--Calendar View (Visual overview)
```

Step 3: Apply UI Design Principles

| Principle | Implementation |
|---|---|
| Visibility | Dashboard shows today's activity and alerts at a glance |
| Feedback | Confirmation screen with checkmark; color-coded calendar |
| Constraints | Unavailable rooms are grayed out and unselectable |
| Consistency | Same button placement, colors, and navigation across screens |
| Affordance | Buttons look clickable; radio buttons for single selection |

Step 4: Design Individual Screens

| Screen | Design Decisions |
|---|---|
| Dashboard | Large action buttons for common tasks; alerts section highlights urgent items (pending guarantees) |
| Date Selection | Calendar widget for visual date picking; shows selected range highlighted |
| Room Selection | Card-based layout shows all room details at once; unavailable rooms visible but disabled |
| Guest/Payment | Two-column layout separates reservation summary from data entry; existing guest search saves time |
| Confirmation | Clear success indicator; all details visible; multiple next-action options |
| Manage Guests | Table format for quick scanning; search and pagination for scalability |
| Calendar View | Grid shows room availability across time; color coding for reservation status |

User Interface Design Rationale

-Wizard-style reservation flow: Breaking the process into three steps (dates, room, guest/payment) reduces cognitive load and minimizes errors. John or Jane can focus on one task at a time while on the phone with guests, and the system validates input before allowing progression.

-Dashboard alerts section: This addresses the requirement that unguaranteed reservations must be tracked. Displaying pending guarantee deadlines prominently ensures John and Jane will not miss a deadline and lose a potential booking. Alerts are sorted by urgency with the most imminent deadlines first.

-Color-coded calendar view: Green indicates guaranteed reservations while orange indicates pending reservations needing guarantee payment. This visual distinction allows quick assessment of room availability and identification of bookings needing attention without clicking into details.

-Search existing guest feature: Returning customers are common in the B&B industry. Rather than re-entering all information for repeat visitors, searching by name and selecting an existing record speeds up the process and ensures data consistency.

-Guarantee now vs. later options: Radio buttons for these options directly implement the requirement that reservations can be held without immediate guarantee. When "guarantee later" is selected, the system prompts for a deadline date, accommodating guests who need to confirm travel plans first.

-Card-based room selection: Displaying room name, features, nightly rate, and total price in a single view supports the phone-based workflow. John or Jane can quickly relay all options to the guest without navigating between screens. Unavailable rooms are grayed out but visible, providing context while preventing booking errors.

-Back buttons on every screen: This allows users to correct mistakes without losing progress, which is important during phone conversations where guests might change their mind about dates or room preferences. The wizard preserves previously entered information during backward navigation

**7) Rubric Criteria: Reflect on the learning experience and lessons learned 10%**
**Your Response:**

-Importance of Traceability

Every design decision should trace back to a requirement. For example, the guaranteeDeadline attribute in the Reservation class directly implements the requirement that reservations will be held without guarantee for an agreed upon time.

-Layered Architecture Simplifies Complexity

3 tier architecture separates apprehensions. Changes to the UI don't affect business logic, database technology can be swapped without rewriting the application, and each layer can be tested independently. This was evident when designing the class diagram services in the business layer handle rules like "drop expired reservations," keeping that logic out of the UI.

-Design Models Bridge Analysis and Implementation

The analysis class diagram showed what entities exist. The design class diagram added how they work:

Data types for attributes
Method signatures
Visibility modifiers
Detailed relationships

This translation from analysis to design is where engineering decisions happen.

-UI Design Must Support User Workflows

Designing the UI forced me to think about how the users work. If the users are on the phone with guests they will need quick access to availability. To quickly track guarantee deadlines, dashboard alerts are an efficient feature. To quickly find returning guests, a search existing guest feature will help the users.

The UI isn't just about making things look nice, it's about optimizing task completion.